

Rapport d'Analyse de l'Algorithme de Routage pour Itinéraires Cyclables avec Contraintes Multiples

- L'algorithme vise à trouver 3 itinéraires cyclables diversifiés entre deux points, respectant des contraintes doubles sur longueur, dénivelé, popularité moyenne et largeur moyenne des routes.
- Il utilise une approche bidirectionnelle avec graphe de surcouche basé sur des cellules/portails, et une recherche locale k-hop pour découvrir des raccourcis intra-cellulaires.
- Les contraintes moyennes (popularité, largeur) sont transformées en sommes additives pondérées par la longueur, permettant une gestion exacte et efficace.
- L'élagage par récupérabilité évite l'élimination prématurée de labels potentiellement faisibles, améliorant la complétude de la recherche.
- La politique de labels représentatifs limite la complexité en conservant un nombre borné de labels par portail, sélectionnés selon des critères de diversité et de qualité.

Introduction et Contexte

Objectif de l'algorithme

L'objectif principal de l'algorithme est de calculer **trois itinéraires cyclables diversifiés** entre deux points (par exemple Paris → Orsay), en respectant simultanément **quatre contraintes doubles** :

- **Longueur** : entre 30 et 35 km.
- **Dénivelé positif** : entre 400 et 500 mètres.
- **Popularité moyenne** : supérieure ou égale à 150.
- **Largeur moyenne des routes** : inférieure ou égale à 10, avec une préférence pour les petites routes.

La diversité des solutions est essentielle pour offrir aux utilisateurs des alternatives variées, répondant à différents compromis entre ces contraintes. Par ailleurs, l'algorithme doit être **efficace computationnellement**, en évitant un prétraitement lourd du graphe, ce qui est crucial pour des applications réelles sur de grands réseaux.

Approche Générale

L'algorithme repose sur une **architecture bidirectionnelle** combinant :

- Un **graphe de surcouche** (*overlay*) basé sur une partition du réseau en **cellules/portails**, permettant de réduire la complexité de la recherche globale.



- Une **recherche locale k-hop** pour explorer les raccourcis intra-cellulaires, déclenchée à la demande pour éviter un prétraitement coûteux.
- Une gestion des contraintes via des **transformations additives** qui convertissent les contraintes moyennes (popularité, largeur) en sommes pondérées par la longueur, facilitant ainsi leur intégration dans le calcul des coûts.
- Une stratégie d'**élagage par récupérabilité** (*recoverability pruning*) pour éviter d'éliminer prématurément des labels qui pourraient encore mener à des solutions faisibles.
- Une politique de **labels représentatifs** limitant le nombre de labels conservés par portail, pour maîtriser la complexité tout en conservant la diversité des solutions.

Points Forts de l'Algorithme (À Conserver)

Transformation Additive des Contraintes Moyennes

Méthode : L'algorithme transforme les contraintes moyennes (popularité et largeur) en sommes additives pondérées par la longueur des segments. Par exemple, la contrainte sur la popularité moyenne est convertie en :

$$\text{popDef} = \sum (P_{\min} - p(e)) \cdot \ell(e)$$

où $P_{\min}$ est la popularité minimale souhaitée, $p(e)$ la popularité de l'arête e , et $\ell(e)$ sa longueur.

Avantages :

- Cette transformation permet une gestion exacte des contraintes moyennes, en tenant compte de la longueur des segments, ce qui est plus précis que de considérer une moyenne par arête.
- Elle facilite la mise à jour incrémentielle des coûts lors de l'expansion des labels, ce qui est crucial pour l'efficacité.
- Elle est compatible avec la gestion des raccourcis locaux, car les coûts additifs peuvent être calculés exactement pour les sous-chemins.

Recommandation : Conserver cette approche, mais **documenter clairement** l'hypothèse de pondération par la longueur, car les utilisateurs pourraient s'attendre à une moyenne par arête. Il serait utile d'ajouter un paramètre pour choisir le type de moyenne (pondérée par longueur ou par nombre d'arêtes) ¹.

Élagage par Récupérabilité (*Recoverability Pruning*)

Méthode : Avant d'élaguer un label, l'algorithme vérifie si une solution partielle peut encore satisfaire les contraintes en utilisant le budget restant. Par exemple, pour la popularité, on calcule :

$$\text{bestAddPopDef} = (P_{\min} - 255) \cdot L_{\text{remMax}}$$

où L_{remMax} est la longueur maximale restante.



Avantages :

- Évite l'élagage prématuré des labels qui pourraient devenir faisables plus tard, améliorant ainsi la complétude de la recherche.
- Le calcul est peu coûteux et s'intègre naturellement dans la boucle principale.
- Permet de conserver des labels qui semblent infaisables mais qui pourraient être récupérés via des raccourcis locaux.

Recommandation : Conserver cette stratégie, mais valider les bornes utilisées (ex. $P_{\min} = 150$) pour éviter des corrections trop optimistes qui pourraient fausser la faisabilité ¹.

Labels Représentatifs (vs. Dominance de Pareto)

Méthode : Plutôt que de conserver tous les labels dominants au sens de Pareto, l'algorithme maintient un ensemble borné de labels représentatifs par portail (typiquement 8 à 12), sélectionnés selon des critères de slots (meilleur score, longueur min/max, dénivelé min/max).

Avantages :

- Évite l'explosion combinatoire due à la dominance classique, qui est inefficace avec des contraintes doubles.
- Permet de conserver des labels « lourds » en ressources mais utiles pour atteindre les bornes inférieures.
- Réduit la complexité tout en conservant la diversité nécessaire pour couvrir les contraintes.

Recommandation : Conserver cette politique, mais **ajuster dynamiquement** la taille des ensembles en fonction de la densité des portails et de la complexité du graphe ².

Construction Incrémentale des Raccourcis Locaux (Lazy k-hop)

Méthode : Les raccourcis intra-cellulaires sont construits **à la demande**, déclenchés par des labels prometteurs, et non pas en prétraitement.

Avantages :

- Évite un prétraitement coûteux et inadapté aux contraintes dynamiques.
- Adapte l'effort computationnel aux zones pertinentes du graphe.
- Permet de découvrir des raccourcis locaux qui améliorent la qualité des solutions sans surcharge initiale.

Recommandation : Rendre cette construction **incrémentale** (contexte local persistant) pour éviter les recalculs redondants et améliorer l'efficacité ².



Couplage Bidirectionnel et Graphe de Surcouche

Méthode :

- Recherche bidirectionnelle avec états forward/backward.
- Graphe de surcouche limité aux portails + raccourcis locaux.
- Jonction des solutions au niveau des portails communs.

Avantages :

- Réduction significative de l'espace de recherche.
- Flexibilité pour ajouter des portails dynamiquement si nécessaire.
- Permet de gérer efficacement les contraintes multiples en limitant la complexité.

Recommandation : Conserver cette architecture, mais **valider la symétrie** pour les graphes dirigés (raccourcis entrants pour la recherche backward) ^{3 2}.

Risques et Améliorations Clés

Définition Explicite des Moyennes

Problème : L'algorithme suppose une **moyenne pondérée par la longueur**, mais cela doit être **clairement documenté et validé**.

Risque : Si les utilisateurs s'attendent à une moyenne par arête, les résultats pourraient être biaisés.

Solution :

- Ajouter un paramètre `averageType` (ex. `LENGTH_WEIGHTED` ou `EDGE_COUNT`).
- Pour une moyenne par arête, stocker (`sumP`, `sumW`, `edgeCount`) et ajuster les mises à jour.

Arrêt Prématuré et Bornes Inférieures

Problème : La condition d'arrêt `bestFrontierKey >= archive.worstScore()` n'est **safe** que si `Key()` est une **borne inférieure admissible** (ce qui n'est pas garanti avec un score Chebyshev).

Risque : Arrêt prématuré avant d'avoir exploré toutes les solutions faisibles.

Solutions :

- **Option 1** : Utiliser un **soft stop** (ex. "aucune amélioration depuis N expansions").
- **Option 2** : Calculer une **borne inférieure admissible** pour `len` (ex. distance euclidienne $\times 1.0$) et l'intégrer dans `Key()`.
- **Option 3** : Désactiver l'arrêt prématuré si le score n'est pas admissible.



Gestion des Arêtes Dirigées et Raccourcis Symétriques

Problème : Si le graphe de base est dirigé (ex. sens uniques), les raccourcis locaux doivent être **cohérents avec la direction d'expansion**.

Risque : Oublier de construire des raccourcis **entrants** pour la recherche backward.

Solution :

- Pour la recherche **backward**, soit :
 1. **Inverser les arêtes** du graphe de base pour la recherche locale (coûteux).
 2. **Construire des raccourcis entrants** ($u \rightarrow \text{portail}$) via une recherche locale "vers le portail" (recommandé).

Exemple de pseudocode pour **GrowShortcutsToPortal** :

```
function GrowShortcutsToPortal(cellId, targetPortal q, ...):  
    # Lance une recherche locale depuis les portails/vertices vers q  
    # Émet des edges  $u \rightarrow q$  pour la recherche backward
```

Éviter les Recherches Locales Redondantes

Problème : La construction des raccourcis locaux peut être **relancée plusieurs fois** pour le même portail si le budget est épuisé.

Risque : Gaspi computationnel et incohérences si les raccourcis changent entre les appels.

Solution :

- **Contexte local persistant** : Stocker l'état de la recherche locale (PQ, états explorés) pour chaque (cellId, portail, direction).
- **Reprise incrémentale** : À chaque déclenchement, reprendre la recherche là où elle s'était arrêtée.

Exemple :

```
struct LocalSearchContext:  
    pqLocal: MinPQ<(key, LocalStateId)> # États en attente  
    repAtVertex: Map<VertexId, TinyRepSet> # Filtre local  
    foundEdges: Set<OverlayEdge> # Raccourcis déjà émis  
    stepsSpent: int # Budget déjà consommé
```

Compression des Raccourcis par ϵ -Dominance

Problème : Conserver **un seul raccourci** par paire (p, q) peut éliminer des compromis utiles (ex. un raccourci plus long mais avec une meilleure popularité).

Risque : Perte de diversité dans les solutions finales.



Solution :

- Utiliser une **ϵ -dominance** pour conserver **2-4 variants** par paire (p, q).
- Normaliser les coûts (len, gain, popDef, widthExc) et regrouper en bins.

Exemple :

```
function insertEpsilonDominance(edgeSet, newEdge,  $\epsilon$ , maxSize):
    normalizedCosts = normalize(newEdge.cost)
    for existing in edgeSet:
        if  $\epsilon$ Dominated(existing, newEdge,  $\epsilon$ ):
            return False # newEdge est dominé
        if  $\epsilon$ Dominated(newEdge, existing,  $\epsilon$ ):
            edgeSet.remove(existing) # existing est dominé
    if len(edgeSet) < maxSize:
        edgeSet.add(newEdge)
        return True
    return False
```

Filtre de Compatibilité pour la Jonction

Problème : La jonction de labels forward/backward au même portail peut être coûteuse si tous les paires sont testées.

Risque : Jusqu'à $\text{maxRepsPerPortal}^2$ combinaisons (ex. 144 pour 12 labels).

Solution :

- **Pré-filtre rapide** avant la jonction :

```
if (Lf.len + Lb.len > Lmax) or (Lf.gain + Lb.gain > Gmax):
    continue
if RecoverablyInfeasible(AddCost(Lf.cost, Lb.cost), C):
    continue
```

- **Tri des labels opposés** par compatibilité (ex. $\text{abs}(Lf.len - Lb.len)$) pour limiter les tests.

Clé Heuristique pour la PQ (Key)

Problème : La fonction **Key()** actuelle (basée sur Chebyshev) peut ne pas être **admissible**, ce qui rend l'arrêt prématuré unsafe.

Solution :

- **Option 1** : Utiliser une **borne inférieure admissible** pour len (ex. distance euclidienne) + terme heuristique pour les autres attributs.



```
function Key(label, dir, C, heur):
    admissibleBound = label.cost.len + straightLineDistance(label.portal, target)
    # Ajouter des termes pour gain/popDef/widthExc (non admissibles mais utiles)
    return admissibleBound + heur.weightPop * label.cost.popDef + ...
```

- **Option 2** : Séparer la clé en une **partie admissible** (pour l'élagage safe) et une **partie heuristique** (pour le tri).

Pseudocode Révisé et Fonctions Clés

Fonctions de Coût et Faisabilité

```
# Mise à jour des coûts pour une arête de base
function BaseEdgeDeltaCost(e, C) -> (Cost, RoadId):
    Δlen = e.ℓ
    Δgain = e.g
    ΔpopDef = (C.Pmin - e.p) * e.ℓ
    ΔwidthExc = (e.w - C.Wmax) * e.ℓ
    ΔroadChg = 1 if (lastRoadId != e.roadId and lastRoadId != null) else 0
    return Cost(Δlen, Δgain, ΔpopDef, ΔwidthExc, ΔroadChg), e.roadId

# Vérification de faisabilité globale
function Feasible(cost, C) -> bool:
    return (C.Lmin ≤ cost.len ≤ C.Lmax and
            C.Gmin ≤ cost.gain ≤ C.Gmax and
            cost.popDef ≤ 0 and
            cost.widthExc ≤ 0)
```

Boucle Principale Bidirectionnelle (Extraits Clés)

```
while not (pqF.empty() and pqB.empty()):
    dir = FWD if pqF.peekKey() ≤ pqB.peekKey() else BWD
    (key, id) = pq[dir].pop()
    L = LabelStore[id]

    if HardInfeasible(L.cost, C) or RecoverablyInfeasible(L.cost, C):
        continue

    # Insertion dans les représentants
    if not RepSetInsert(rep[dir][L.portal], L, cfg.maxRepsPerPortal):
        continue

    # Jonction au portail
    if L.portal in rep[oppDir]:
        for R in RepSetGet(rep[oppDir][L.portal]):
            totalCost = AddCost(L.cost, R.cost)
            totalCost.roadChg += StitchRoadChange(L.cost.lastRoadId, R.cost.firstRoadId)
            if Feasible(totalCost, C):
                sol = BuildSolution(L, R, totalCost)
```



```

ArchiveTryInsert(archive, sol, cfg)

# Construction incrémentale des raccourcis
if ShouldTriggerLocal(L, archive, cfg):
    EnsureCellShortcuts(Overlay, Cells, Cells.cellOfPortal(L.portal), L.portal, dir, L.cost, C)

# Expansion via les arêtes de surcouche
for E in (Overlay.outEdges(L.portal) if dir == FWD else Overlay.inEdges(L.portal)):
    newCost = CombineCosts(L.cost, E.cost, dir)
    if not (HardInfeasible(newCost, C) or RecoverablyInfeasible(newCost, C)):
        newLabel = NewLabel(..., newCost, ...)
        pq[dir].push(Key(newLabel, dir, cfg), newLabel.id)

```

Construction Incrémentale des Raccourcis Locaux

```

function EnsureCellShortcuts(Overlay, Cells, cellId, p, dir, prefixCost, cfg):
    ctx = Overlay.getLocalCtx(cellId, p, dir)
    if not ctx or Overlay.shortcutDegree(cellId, p, dir) < cfg.minShortcutsPerPortal:
        newEdges = LocalSearchStep(ctx, cfg.localStepBudget, cfg.kHopLimit, prefixCost, C)
        for E in newEdges:
            Overlay.addEdge(E)

function LocalSearchStep(ctx, stepBudget, kHopLimit, prefixCost, C) -> list[OverlayEdge]:
    if not ctx.initialized:
        ctx.initialized = true
        s0 = NewLocalState(p.vertex, ZeroCost(), ...)
        ctx.pqLocal.push(0, s0.id)

    emitted = []
    expansions = 0
    while not ctx.pqLocal.empty() and expansions < stepBudget:
        S = ctx.pqLocal.pop()
        if S.hops > kHopLimit: continue

        if Cells.isPortalVertex(S.vertex) and S.vertex != p.vertex:
            q = Cells.portalOfVertex(S.vertex)
            E = BuildShortcutEdge(p, q, dir, S.cost, ...)
            if ctx.foundByDest[q].insertEpsilonDominance(E, cfg.ε, cfg.maxShortcutsPerPair):
                emitted.append(E)

        for e in Gbase.outEdges(S.vertex): # ou inEdges si dir == BWD
            if Cells.cellOfVertex(e.to) != cellId: continue
            newState = ExtendLocalState(S, e, C)
            if PruneLocal(newState, prefixCost, C, kHopLimit):
                continue
            ctx.pqLocal.push(LocalKey(newState, cfg), newState.id)
            expansions += 1

    return emitted

```



Archivage des Solutions avec Diversité

```
class SolutionArchive:
    function TryInsert(solution):
        for existing in archive:
            if Similarity(existing, solution) > cfg.simThresh:
                if solution.score < existing.score - cfg.improvementMargin:
                    archive.replace(existing, solution)
                return False
        if len(archive) < cfg.K or solution.score < archive.worstScore():
            archive.add(solution)
            archive.sort()
            if len(archive) > cfg.K: archive.pop()
        return True

    function Similarity(solA, solB) -> float:
        # Option 1: Overlap des arêtes
        return len(solA.edges ∩ solB.edges) / min(len(solA.edges), len(solB.edges))
        # Option 2: Overlap des road_id (plus stable)
        return len(solA.roadIds ∩ solB.roadIds) / min(len(solA.roadIds), len(solB.roadIds))
```

Recommandations d'Implémentation

Paramètres par Défaut pour une Première Implémentation

Paramètre	Valeur Sugérée	Justification
kHopStart	20	Suffisant pour la plupart des cellules.
kHopMax	80	Limite pour éviter l'explosion.
localStepBudget	300	Budget par déclenchement.
maxRepsPerPortal	10	Équilibre entre diversité et efficacité.
maxShortcutsPerPair	3	2-4 variants par paire (p, q).
minShortcutsPerPortal	6	Évite de relancer trop souvent la recherche.
simThresh (diversité)	0.7	Seuil pour l'overlap des arêtes/road_ids.
ϵ (pour ϵ -dominance)	0.1	Tolérance pour la compression des raccourcis.

Métriques de Diversité

- **Préférer l'overlap des road_id** à celui des arêtes :
 - Plus stable (moins sensible aux petits détours).
 - Mieux aligné avec l'objectif de "suivi de routes lisses".



- **Alternative** : Combiner plusieurs métriques (ex. overlap des arêtes + différence de longueur).

Calcul du Score (Chebyshev-to-Center)

Formule suggérée :

```
function Score(cost, C, weights) -> float:
    # Normaliser chaque composante entre [0, 1] (0 = meilleur)
    normLen = (cost.len - C.Lcenter) / (C.Lmax - C.Lcenter) # Lcenter = (Lmin + Lmax)/2
    normGain = (cost.gain - C.Gcenter) / (C.Gmax - C.Gcenter)
    normPop = max(0, cost.popDef) / popDefMaxPossible # popDef ≤ 0 est feasible
    normWidth = max(0, cost.widthExc) / widthExcMaxPossible
    # Score Chebyshev (max des écarts normalisés)
    chebScore = max(normLen, normGain, normPop, normWidth)
    # Ajouter un terme pour la fluidité (road changes)
    smoothTerm = cost.roadChg / maxRoadChanges # normalisé
    return chebScore + weights.smooth * smoothTerm
```

- **Poids suggérés** : `weights.smooth = 0.2` (à ajuster selon l'importance de la fluidité).

Clé pour la PQ (Key)

Version admissible + heuristique :

```
function Key(label, dir, C, heur):
    # Borne inférieure admissible pour la longueur
    admissibleLen = label.cost.len + straightLineDistance(label.portal, target[dir])
    # Terme heuristique pour les autres attributs (non admissible mais utile)
    heurTerm = (heur.wPop * label.cost.popDef +
                heur.wWidth * label.cost.widthExc +
                heur.wGain * abs(label.cost.gain - C.Gcenter))
    return admissibleLen + heurTerm
```

- **Poids heuristiques** : `wPop = 0.5`, `wWidth = 0.3`, `wGain = 0.2` (à calibrer).

Conclusion et Prochaines Étapes

Synthèse des Points Clés

- **À conserver** :
 - Transformation additive des contraintes moyennes.
 - Élagage par récupérabilité et labels représentatifs.
 - Construction lazy des raccourcis locaux.
- **À améliorer** :
 - **Incrémentalité** des recherches locales (contexte persistant).
 - **Symétrie** pour les graphe dirigés (raccourcis entrants).



- **Compression ε -dominante** des raccourcis.
- **Clé admissible** pour la PQ et arrêt prématuré safe.

Prochaines Étapes Recommandées

1. **Implémenter le contexte local persistant** pour éviter les recalculs.
2. **Valider la symétrie** des raccourcis pour les graphe dirigés.
3. **Calibrer les poids** du score Chebyshev et de la clé Key ().
4. **Tester avec des instances réelles** (ex. Paris → Orsay) pour :
 - Vérifier la diversité des solutions.
 - Mesurer l'impact des améliorations (temps CPU, qualité des solutions).

Extensions Futures

- **Parallélisation** : Les recherches locales dans différentes cellules peuvent être parallélisées.
- **Heuristiques apprises** : Remplacer les poids manuels dans Key () par un modèle appris (ex. régression sur des trajets connus).
- **Adaptation dynamique** : Ajuster kHopLimit ou maxRepsPerPortal en fonction de la densité locale du graphe.

Ce rapport fournit une analyse complète de l'algorithme proposé, en mettant en lumière ses forces, ses risques, et des pistes concrètes pour son amélioration et son implémentation. Les sections sont organisées pour faciliter la priorisation des tâches, depuis les ajustements critiques (ex. incrémentalité des recherches locales) jusqu'aux optimisations optionnelles (ex. parallélisation).

-
- [1] [Mastering Constraint Handling in Optimization: A Deep Dive into Transformation Techniques](#)
 [2] <https://graal.ens-lyon.fr/~abenoit/reso05/cours/1-routage.pdf>
 [3] [Conception des algorithmes pour le routage à multiples contraintes](#)

